Pamantasan ng Lungsod ng Maynila

AN ENHANCEMENT OF FAST K-MEANS ALGORITHM APPLIED IN WEB APPLICATION ON IMAGE COLOR PALETTE GENERATION

A Thesis Presented to the
Faculty of Computer Science Department
College of Information System and Technology Management
Pamantasan ng Lungsod ng Maynila

In Partial Fulfillment of the Requirements for the Degree
Bachelor of Science in Computer Science

By
Atencia, Dan Erdz B.
Dela Cruz, John Bryan P.

December 2025

TABLE OF CONTENTS

TABLE OF CONTENTS i
LIST OF FIGURES ii
INTRODUCTION 1
1.1 Background of the study 1
1.1.1 Existing Algorithm 2
1.2 Statement of the Problem 3
1.3 Objective of the study 7
1.4 Significance of the Study 7
1.5 Scope and Limitations 8
1.6 Definition of terms 9
REVIEW OF RELATED LITERATURE 10
2.1.1 Overview of Boyer-Moore Horspool-Sundays 2 Algorithm 10
2.1.2 Hash-Based Preprocessing and Data Structures 10
2.1.3 GPU-Based Acceleration for Large-Scale Logs 10
2.1.4 Applications in Legal and Structured Text Retrieval 11
2.1.5 Web-Based Retrieval Systems 11
2.1.6 Hybrid and Adaptive Shifting Mechanisms 11
2.1.7 Memory Management in Resource-Constrained Systems 12
2.1.8 Importance of Complete Pattern Detection 12
2.1.9 The Demand for Scalable Blockchain Forensics 12
2.1.10 Lightweight Techniques for Blockchain Search and Detection 12
2.1.11 Pattern Matching Efficiency in Ethereum Virtual Machine 13
2.1.12 Modern Blockchain Forensics and Crime Investigation Frameworks 13
2.1.13 Pattern Matching Rules for Detecting Illicit Bitcoin Transactions ... 14
THEORETICAL FRAMEWORK 15
3.1 Research Design 15
3.2 Conceptual Framework 15
3.3 Proposed Algorithm 16
3.4 System Requirements 16
3.5 Methodology 17
LIST OF REFERENCES 21

LIST OF FIGURES

Figure No.	Figure Title	Page
Figure 1	Peak memory of BMHS2 …….....	4
Figure 2	Peak memory of BHMS2 in various dataset sizes……	4
Figure 3	Execution time of BMHS2 in different pattern lengths……	5
Figure 4.1	910 found of “0xfa4dfbe7b012b1d92” in dataset…….	6
Figure 4.2	45 matches of “0xfa4dfbe7b012b1d92” from running BMHS2……	6

Chapter One

INTRODUCTION

This chapter introduces the research conducted on enhancing the Fast K-Means algorithm for color quantization. The core focus is improving the algorithm's color palette generation.

1.1 Background of the Study

The Fast K-Means algorithm, is an accelerated variant of the classical K-Means clustering technique designed to significantly reduce the computational burden associated with grouping large datasets. It achieves its speed advantage primarily through a two-stage process that first selects a highly representative initial palette or set of centroids and then refines these centers using a highly efficient K-Means iteration stage that incorporates data sampling (Huang, 2021). This technique ensures rapid convergence to optimized cluster centers while minimizing the need to process every data point in every iteration, making it ideal for real-time applications.

In computer graphics, color quantization or color image quantization is a process that reduces the number of distinct colors present in an image, usually with the intention that the new image should be as visually similar as possible to the original image. Various methods are available for color quantization such as the median cut algorithm, popularity algorithm (Aditya, 2015). Color quantization is necessary for displaying images on devices that have a limited color palette, such as older computer monitors or mobile screens. It is also used in image file formats, like GIF and PNG, which can only store a limited number of colors per pixel. The process involves defining a new, smaller color map (palette) and then mapping each pixel's original color to the closest color in the new palette. The choice of quantization algorithm significantly impacts the quality of the resulting image.

Color palette generation from digital images is the process of transforming a vast array of colors within an image into a practical, smaller set, primarily by identifying the image's dominant colors. This process is crucial because color palettes are a critical component of art and design applications, providing the basis for organizing and utilizing colors to achieve specific objectives. Appropriate color design can significantly increase a product's added value, and color is considered the most critical factor in the design process. Generating these palettes is beneficial because it not only assists in diverse tasks like web and product design, film color grading, image classification, and compression, but it also helps to balance the image quality with efficiency in storage and transmission. Algorithms like K-means clustering are key to this, as they automatically extract these dominant colors, overcoming the limitations of manual selection, which can be time-consuming, subjective, and difficult to scale (Yafan, 2024).

The core objective of this research initiative is to undertake a comprehensive enhancement of the existing Fast K-Means clustering algorithm. This enhancement is specifically targeted at the domain of color quantization and palette generation. The ultimate goal is to leverage the improved computational efficiency and clustering quality inherent in the modified algorithm to consistently generate a color palette that is demonstrably superior in terms of aesthetic quality, representativeness of the source image, and overall visual harmony when compared to palettes produced by the standard Fast K-Means or other conventional color clustering methods.

1.1.1 Existing Algorithm

Stage 1 Initial Palette Generation

Step 1: The initial palette is an empty set.

Selected(i) = 0, i = 0, 1, . . . , N−1. The value of Selected(i) is equal to 1 which indicates that the color initc(i) has been selected and added to the initial palette. Cno = 0.

Step 2: If initn(j) == max {initn(i) | i = 0, 1, . . . , N−1}, then the color initc(j) is selected and added to the initial palette. Selected(j) = 1 and Cno = Cno +1.

Step 3: For the colors initc(i) with Selected(i) = 0, compute the square Euclidean distance between the color initc(i) and each color in the initial palette. The minimum value among these square Euclidean distances is identified as Dist(i). For the color initc(i), the value DistN(i) is computed as follows:

$$DistN(i) = Dist(i) \times (initn(i))^{0.5}$$

If DistN(j) == max {DistN(i) | if Selected(i) == 0, i = 0, 1, . . . , N−1}, then the color initc(j) is selected and added to the initial palette.

Step 4: If K > Cno, then go to Step 3. Otherwise go to Step 5.

Step 5: The initial palette is generated.

Stage 2 Fast K-Means Algorithm

Input: The initial palette

Step 1: The data sampling process is employed. The sampled color pixels are denoted by SCPi , i = 0, 1, . . . , SPN—1.

Iter = 0

StopF = 0

SOP 2

Step 2: For the sampled pixel SCPi , the closest color in the palette is denoted by CCPi . The color CCPi is efficiently determined by the Wu–Lin method. If the index value of its closest palette color is r, then the sampled pixel is classified into the r-th group. **SOP 1**

Step 3: Compute the mean values of these K groups. The new color palette is generated by sorting these K values in ascending order of length. **SOP 3**

Step 4: The mean square error at iteration Iter is defined as follows:

$$MSE1(Iter) = \frac{1}{SPN} \sum_{i=0}^{SPN-1} SED(SCP_i, CCP_i).$$

If ((Iter >0) and (MSE1(Iter) ≥ MSE1(Iter − 1))), then set StopF to 1. **SOP 1**

Step 5: Iter = Iter + 1. If ((Iter == Max_cycle) or (StopF == 1)), then go to Step 6. Otherwise, go to Step 2.

Step 6: The color palette is generated. The required iteration number, denoted by Iter, is output.

1.2 Statement of the Problem

This study focuses on enhancing the Fast K-Means algorithm by addressing the following key challenges:

1. **The existing algorithm measures distortion using RGB mean squared error, despite RGB being device-dependent and perceptually non-uniform.**

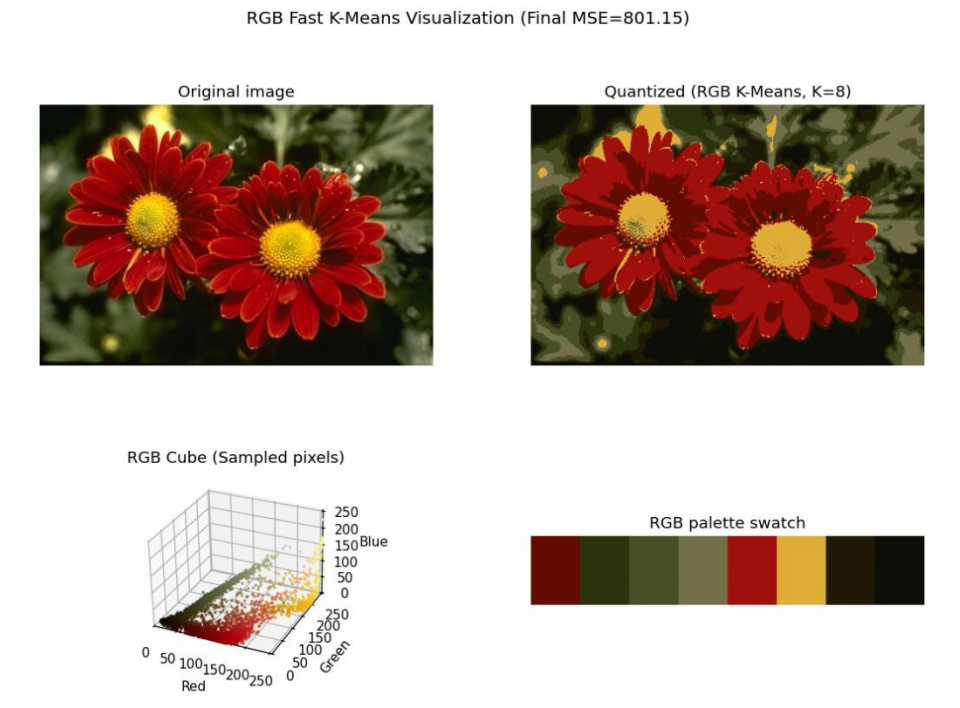


Figure 1. *Example palette produced by the RGB-Euclidean distance.*

In color-based image processing, several studies have reported that the choice of color space has a strong impact on perceptual quality. Arumugadevi and Seenivasagam (2015) compare clustering methods for color image segmentation and emphasize that RGB is device-dependent and not efficient for robust color specification and recognition, while CIE L*a*b* is designed to approximate human vision and is device-independent. Their FCM-based method therefore converts images from RGB to CIE L*a*b* before clustering and achieves better segmentation quality in this perceptual space compared to directly using RGB.

2. **The existing algorithm is effectively limited to images 512×512 or below, because its clustering repeatedly performs pixel-to-palette distance**

assignments and mean updates whose runtime grow rapidly with image size, making larger resolutions impractical for real-time execution.

Figure 2. *Baseline on full-resolution input demonstrates the practical 512×512 limit, as runtime scales with pixel count. (19,961,856 pixels)*

In efficient palette generation for color image quantization, Huang (2021) refines the palette in Stage 2 using Fast K-Means, where each iteration assigns pixels to the nearest palette color and then updates cluster means. The method also includes a data sampling process where a sampling rate of 1 uses all pixels plus the Wu–Lin fast nearest-color search to reduce computation time. Because the assignment-and-update cycle is repeated across iterations and the number of comparisons depends on how many pixels are processed, higher-resolution images naturally require more computation, which motivates practical processing caps (e.g., ≤512×512) for real-time execution.

3. **The existing algorithm lacks a perceptual separation metric, so it can assign two numerically different but visually indistinguishable colors to separate clusters, wasting valuable slots in the limited palette.**

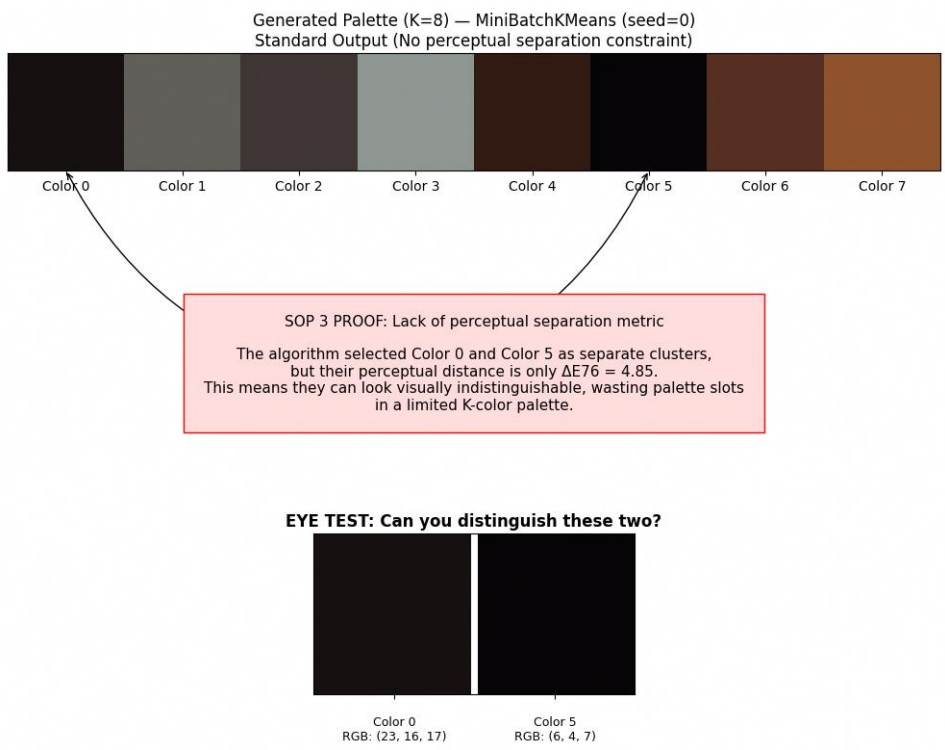


Figure 3. *Near-duplicate palette colors caused by no perceptual separation constraint*

In a recent study on accessible color sequences for data visualization, Petroff (2024) explicitly states that when building a set of colors, a minimum perceptual distance between colors should be enforced to ensure the colors remain distinguishable. He computes perceptual distance in a perceptually uniform space (CAM02-UCS) and applies minimum-perceptual-distance constraints during color-set generation to prevent two selected colors from becoming too similar in appearance.

1.3 Objective of the Study

1.3.1 General Objective

This study focuses on enhancing the Fast K-Means algorithm with the specific goal of optimizing the color palette generation process from digital images.

1.3.2 Specific Objectives

1. **To integrate a color space that maintains visual consistency and aesthetic appeal for the human eye perception.**
2. **To improve the scalability and computational efficiency of the Fast K-Means algorithm for processing higher-resolution images beyond 512×512 while maintaining real-time feasibility.**
3. **To enforce perceptual separation among palette colors by detecting and resolving near-duplicate centroids, ensuring each palette slot represents a visually distinct color.**

1.4 Significance of the Study

The enhancement of the Fast K-Means algorithm for color palette generation will provide significant benefits across various applications:

Computer Graphics and Design: It will provide artists, designers, and developers with a more accurate and perceptually optimized tool for extracting dominant colors, leading to higher-quality, more harmonious color palettes for web design, product design, and image color grading.

Web and Product Design: The enhanced algorithm will automatically generate higher-quality color palettes that more accurately reflect the image's dominant, visually distinct colors. This will provide designers with a more effective and time-saving tool for web development, film color grading, and creating products with increased aesthetic value.

Real-Time Applications: The Fast K-Means algorithm is already designed for efficiency.

Enhancing its output quality while maintaining its speed will make it ideal for real-time applications where both rapid computation and high visual quality are required.

Image File Formats: The refined color quantization process will be beneficial for image formats like GIF and PNG, ensuring that the reduced color set maintains maximum visual similarity to the original image.

Researchers: This study contributes to the field of color quantization and clustering by demonstrating the practical necessity of using perceptually uniform color spaces and separation metrics in image processing algorithms.

1.5 Scope and Limitations

This study focuses on enhancing the Fast K-Means algorithm for image color palette generation and implementing it in a web-based application that accepts user-uploaded images and outputs a compact, visually representative palette. The enhancement work centers on improving palette quality and practicality by incorporating perceptual processing during refinement (e.g., clustering behavior and palette distinctness), and evaluating performance using measurable indicators such as runtime, mean squared error (MSE), and perceptual palette behavior under controlled experiments on a standardized dataset.

This study is limited to two prevalent and widely-used raster image formats JPG and PNG specific image formats only. It does not apply to any other formats other than image formats specified. These two formats were selected due to their widespread adoption in digital photography, web design, and general image storage. The limitations explicitly

exclude, but are not restricted to, formats such as: GIF (Graphics Interchange Format), TIFF (Tagged Image File Format), BMP (Bitmap), WebP, RAW formats (e.g., NEF, CR2, ARW), Vector graphic formats (e.g., SVG, AI, EPS). Therefore, any interpretations or generalizations of this study's results must be made with the explicit acknowledgment that the investigation was meticulously tailored and restricted solely to the specific characteristics, compression algorithms, and metadata structures inherent to the JPG and PNG image formats.

1.6 Definition of Terms

Centroid – The representative "center" color of a cluster, computed as the mean of all pixels assigned to that cluster.

CIELAB (CIE L*a*b*) – A device-independent, more perceptually meaningful color space where distances better reflect human-perceived color differences.

Cluster – A group of pixels assigned to the same palette color/centroid during K-Means grouping.

Color Palette – A limited set of representative colors extracted from an image, typically used to summarize dominant/important colors for design or compression tasks.

Color Quantization – The process of reducing the number of distinct colors in an image by mapping each pixel to the closest color in a smaller palette.

Fast K-Means – An accelerated variant of K-Means used in color quantization that speeds up clustering by using sampled pixels and efficient nearest-color search, then iteratively updates cluster means until convergence.

JPG (JPEG) – A common raster image format that uses lossy compression, widely used in photography and web images.

K (Number of Palette Colors) – The target number of colors to be produced in the final palette (i.e., number of clusters).

K-Means Clustering – A partitioning method that groups data points into K clusters by repeatedly assigning points to the nearest centroid and recomputing centroids as the mean of assigned points.

Mean Squared Error (MSE) – The error measure used to monitor clustering improvement per iteration; in this study context, it is used as a convergence indicator (stop when MSE stops decreasing).

Perceptual Distance (ΔE) – A numerical measure of how different two colors appear to the human eye, typically computed in a perceptual color space such as CIELAB.

Raster Image – An image composed of pixels arranged in a grid as opposed to vector graphics.

PNG – A common raster image format that typically uses lossless compression and supports transparency, widely used in web graphics.

RGB Color Space – A device-oriented color model that represents colors using Red, Green, and Blue channel intensities (commonly 0–255 per channel).

Chapter Two

REVIEW OF RELATED LITERATURE

2.1.1 Overview of Fast K-Means

The Fast K-Means algorithm is an accelerated version of the classical K-Means clustering technique specifically designed to reduce computational cost in color image quantization. This method is the second stage of a two-stage palette generation process. To achieve its speed advantage, the Fast K-Means algorithm employs a data sampling process,

where only a subset of the image's pixels (sampled color pixels, SCPi) is used for clustering. For each sampled pixel, the Wu-Lin method is used to efficiently determine the closest color in the current palette, classifying the pixel into a group. The new color palette is then generated by computing and sorting the mean values of these groups. The algorithm iteratively refines the palette until the Mean Square Error (MSE1(Iter)) ceases to decrease or a maximum cycle count is reached, ultimately producing a color palette for the quantized image (Huang, 2021).

2.1.2 RGB Is Not Perceptually Uniform

In line with our Statement of the Problem 1, Mureika (2005) explicitly points out that the three-dimensional RGB color space is not perceptually uniform, so geometric distances in RGB do not correspond to uniform perceived color differences. This implies that minimizing squared Euclidean error in RGB, as done by the existing fast K-Means palette generation, may produce palettes that are numerically optimal yet visually sub-optimal, supporting our claim that RGB-based error does not reliably reflect perceptual palette quality.

2.1.3 Color Harmony of Palette

Sharma et al. (2023) investigated the subject of color quantization in images, which is well-established; however, the quantization of harmonious color palettes has received less attention. Their research involved applying existing color quantization techniques to crowdsourced color palettes and showed that quantizing the colors within a palette does not significantly impact the perception of harmony. This finding simplifies analytical research into color harmony. Furthermore, the study demonstrated that a basic n-Gram language model, typically used for text prediction, could generate harmonious color combinations with a performance comparable to palettes created by humans.

2.1.4 Palette-Based Image Decomposition

An improved geometric approach for palette extraction, as detailed by Wang et al. (2019), formulates the problem as an optimization that uses a general polyhedron in RGB space. This method balances reconstruction accuracy with representativeness, ensuring the selected palette colors closely match existing pixel colors. For the subsequent layer decomposition, techniques range from slow, optimization-based methods to faster, interpolation-based ones like the RGBXY method. A significant advancement in this area is the use of Mean Value Coordinates (MVCs) for calculating layer opacity. This offers high efficiency and generates spatially smoother layers without requiring pre-computation.

2.1.5 Palette-Based Photo Recoloring

Palette-based photo recoloring, as explored by Chang et al. (2015), presents an intuitive and highly expressive method for image manipulation. This approach effectively balances the simplicity of basic tools (like hue sliders) with the power and complexity of advanced, histogram-based methods (like Photoshop's levels tool). The core of the system is the automatic generation of a small color palette, which condenses the image's primary color groups. Users can then modify the entire image by simply editing the colors in this palette. The main technical hurdle is the color transfer algorithm, which must smoothly interpolate the color changes. Crucially, the algorithm must achieve several goals: maintain

pixel continuity, stay within the color gamut, preserve monotonicity in luminance. Preserving luminance monotonicity is essential to prevent relative brightness from inverting, which would cause undesirable artifacts. To ensure the system is suitable for creative exploration, interactive speed is necessary. This is achieved by accelerating the radial basis function (RBF) color blending process through a fast, table-based method utilizing trilinear interpolation.

2.1.6 Color Quantization Methods

In their 2024 study, Pérez-Delgado and Celebi revealed that the standard practice of evaluating color quantization (CQ) methods primarily with pixelwise error metrics like

MSE, MAE, and PSNR is inadequate. Because these conventional indices ignore the Human Visual System (HVS), they often fail to reflect the actual perceived image quality. The research, comparing ten CQ methods against eight diverse image quality assessment (IQA) indices, demonstrated that different index types produce different performance rankings. Specifically, the simple error metrics (MSE, PSNR), along with Spectral Angle Mapper (SAM), form one correlated group of results, distinct from a second correlated group composed of advanced HVS-based indices: Structural Similarity Index (SSIM), Multi-Scale SSIM (MS-SSIM), and Visual Information Fidelity (VIF). This discrepancy proves that relying on a single pixelwise index can lead to erroneous conclusions about a CQ method's quality. Therefore, the authors strongly recommend that future CQ evaluations abandon the use of a single, pixelwise metric and instead incorporate a suite of IQA indices that capture diverse visual and structural characteristics for a more robust and perceptually relevant quality assessment.

2.1.7 Helminth Parasite Image Segmentation

Khairudin et al. (2020) investigated the use of K-Means (KM) and Fast K-Means (FKM) clustering algorithms for segmenting images of helminth parasites. Segmentation

is crucial for distinguishing the target parasites from artifacts and the background, thereby enabling more accurate detection and classification. Their results demonstrated that FKM clustering yielded superior segmentation performance compared to KM. Specifically, FKM achieved a 3% higher sensitivity than KM. Furthermore, FKM was faster, taking 694 seconds to process 200 helminth parasite images, and showed high performance metrics: 99.33% accuracy, 93.36% sensitivity, and 99.43% specificity. These findings are valuable as they can serve as a reference or a counterpoint for the subsequent classification of helminth species based on their morphology.

2.1.8 Accelerating K-Means Color Quantization via Image Sampling

Frackiewicz et al. (2019) introduced an improved method for image quantization. This technique significantly accelerated the process, sometimes by a few hundred times, without substantially compromising image quality. Notably, their assessment utilized not only the classic Mean Squared Error (MSE) but also the DSCSI and HPSI indices, which are strongly correlated with human visual perception. The enhancement involved applying Nearest Neighbor Interpolation (NNI) for downsampling the input image. Their article demonstrates that as the spatial resolution of the image increases, the outcome of the improved method becomes less dependent on the number of colors in the palette.

2.1.9 Automated Segmentation for Accelerated Nanoparticle Characterization

Day et al. (2025) introduced an efficient, automated image segmentation pipeline designed to overcome a major hurdle in high-throughput nanoparticle characterization. This system is anticipated to be 25.0 to 29.1 times faster than existing baseline methods and functions entirely without human intervention. Key findings from their evaluation of over 900 diverse images indicate a 96.0% success rate, verified against expert-validated standards. The pipeline demonstrates robust performance across varied experimental conditions, effectively managing changes in focus, magnification, and contrast. Furthermore, this approach is expected to substantially conserve storage space by reducing

the required number of acquisition points by an average factor of 29.7. Distinctive elements of the pipeline include the adaptive sizing of acquisition boxes and the incorporation of particle composition data into the segmentation process. The researchers optimized their code for speed and minimized dependencies to guarantee long-term stability and efficiency.

2.1.10 Architectural Color Analysis using K-Means and HSV

The study by Dong and Kang (2025) shifts architectural heritage color analysis from subjective, inefficient methods (like color card comparison) to a modern, quantitative framework using K-means clustering and the HSV color space. K-means efficiently quantifies dominant colors from large image sets, reducing bias, often following pre-processing like semantic segmentation and color correction. Converting the acquired RGB data to HSV is crucial as it aligns with human perception, enabling objective analysis of color distribution. This quantitative approach is vital for identifying regional color characteristics such as hue, saturation, value intervals, informing strategies for architectural color conservation, restoration, and landscape control.

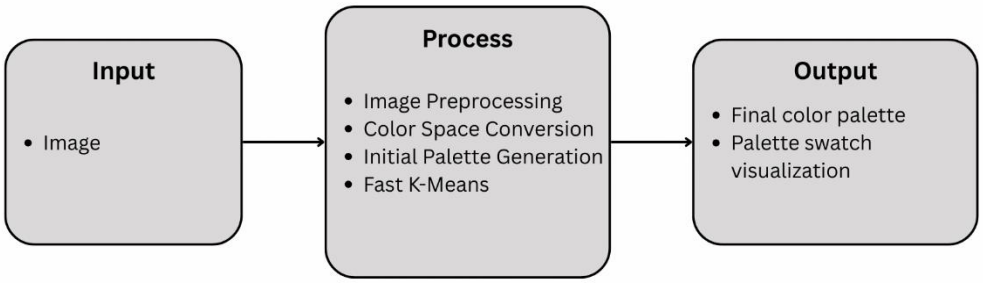
Chapter Three

METHODOLOGY

3.1 Research Design

The research follows an experimental-comparative research design, where the baseline Fast K-Means algorithm is compared against the proposed enhanced version using measurable performance indicators. These indicators include runtime, mean squared error (MSE), pixel count processed, and perceptual palette behavior. The evaluation is conducted through controlled experiments using a standardized image dataset to ensure consistency and reproducibility.

3.2 Conceptual Framework



3.3 Proposed Algorithm

Stage 1 Initial Palette Generation

Step 1: The initial palette is an empty set.

Selected(i) = 0, i = 0, 1, . . . , N−1. The value of Selected(i) is equal to 1 which indicates that the color initc(i) has been selected and added to the initial palette. Cno = 0.

Step 2: If initn(j) == max {initn(i) | i = 0, 1, . . . , N−1}, then the color initc(j) is selected and added to the initial palette. Selected(j) = 1 and Cno = Cno +1.

Step 3: For the colors initc(i) with Selected(i) = 0, compute the square Euclidean distance between the color initc(i) and each color in the initial palette. The minimum value among these square Euclidean distances is identified as Dist(i). For the color initc(i), the value DistN(i) is computed as follows:

$DistN(i) = Dist(i) \times (initn(i))^{0.5}$

If DistN(j) == max {DistN(i) | if Selected(i) == 0, i = 0, 1, . . . , N−1}, then the color initc(j) is selected and added to the initial palette.

Step 4: If K > Cno, then go to Step 3. Otherwise go to Step 5.

Step 5: The initial palette is generated.

Stage 2 Fast K-Means Algorithm

Input: Original image + initial palette

Pre-Step: Resize the original image so max dimension ≤512 while keeping aspect ratio.

Let the resized image be I′

Step 1: The data sampling process is then employed on I'. The sampled color pixels are denoted by SCPi , i = 0, 1, . . . , SPN—1.

Iter = 0

StopF = 0

OBJ 2

Step 2: For the sampled pixel SCPi , the closest color in the palette is denoted by CCPi. Before applying the Wu–Lin method, convert SCP_i and all palette colors into CIELAB coordinates. The closest palette color CCP_i is then determined using the Wu–Lin method based on the CIELAB squared distance. If the index value of its closest palette color is r, then the sampled pixel is classified into the r-th group. **OBJ 1**

Step 3: Compute the mean CEILAB values of these K groups. The new color palette is generated by sorting these K values in ascending order of length.

Step 4: Check the updated palette for near-duplicate colors by measuring perceptual difference (ΔE) in CIELAB. If two palette colors are too similar, keep the one representing more sampled pixels and replace the other with a more distinct candidate color. Repeat until the palette has no near-duplicates, then re-sort the palette by length. **OBJ 3**

Step 5: The mean square error at iteration Iter is defined as follows:

$$MSE1(Iter) = \frac{1}{SPN} \sum_{i=0}^{SPN-1} \|SCP_Lab(i) - CCP_Lab(i)\|^2$$

If ((Iter >0) and (MSE1(Iter) ≥ MSE1(Iter − 1))), then set StopF to 1. **SOP 1**

Step 6: Iter = Iter + 1. If ((Iter == Max_cycle) or (StopF == 1)), then go to Step 6. Otherwise, go to Step 2.

Step 7: The color palette is generated. The required iteration number, denoted by Iter, is output.

3.4 System Requirements

This section presents the hardware and software requirements necessary for the development, execution, and evaluation of the enhanced Fast K-Means color palette

generation system. The system requirements ensure that the proposed algorithm can be implemented efficiently and tested consistently across different image resolutions.

Hardware Requirements:

- Processor: Intel Core i5 or higher
- RAM: 8 GB minimum
- Storage: 256 GB SSD (recommended for faster data loading)

Software Requirements:

- Python 3.10 above
- Text Editor or IDE (VSCode)

Libraries:

- NumPy
- Pillow (PIL)
- OpenCV
- Matplotlib

3.5 Methodology

This section discusses the specific methodologies applied to address the identified statements of the problem in the original Fast K-Means algorithm.

Each supplementary method was carefully designed to resolve a particular problem identified in Chapter 1, while preserving the computational efficiency of the original algorithm. The enhancements focus on perceptual accuracy, representativeness of visually important colors, and robustness against sampling sensitivity.

3.5.1 The existing algorithm measures distortion using RGB mean squared error, despite RGB being device-dependent and perceptually non-uniform.

Theory: The RGB color space does not correspond linearly to human color perception; equal numerical distances in RGB do not translate to equal perceived color differences. Prior studies have shown that RGB-based distance metrics often produce visually sub-optimal results despite low numerical error values. In contrast, the CIE L*a*b* (CIELAB) color space was designed to be perceptually uniform and device-independent, meaning that Euclidean distances in this space better approximate human visual differences between colors. Therefore, using CIELAB for distance computation leads to more perceptually accurate color clustering and palette generation.

Method: To address this limitation, the proposed enhancement replaces RGB-based distance computation with CIELAB-based distance measurement during the Fast K-Means refinement stage. All sampled color pixels and palette centroids are converted from RGB to CIELAB before clustering. The squared Euclidean distance in the CIELAB space is then used to assign pixels to their nearest palette color. Additionally, the mean squared error (MSE) used as the convergence criterion is computed in the CIELAB space rather than in RGB. This ensures that palette refinement is guided by perceptual similarity instead of purely numerical RGB differences.

Tools:

- Python
- NumPy (array math, distance, MSE)

- scikit-image (RGB→CIELAB conversion)
- 3D sphere / color-space visualization (Matplotlib 3D)

3.5.2 The existing algorithm is limited to images of 512×512 resolution or below due to runtime growth

Theory: The computational cost of the Fast K-Means algorithm grows with the number of processed pixels, since each iteration involves repeated pixel-to-centroid distance calculations and centroid updates. For high-resolution images, this leads to excessive runtime that is unsuitable for real-time or interactive applications. Efficient palette generation therefore requires reducing the number of processed pixels without significantly degrading palette quality.

Method: To address the runtime limitation of the original Fast K-Means algorithm on high-resolution images, this study applies resolution-constrained preprocessing prior to palette generation. The input image is first analyzed to determine its spatial resolution. If either dimension exceeds 512 pixels, the image is proportionally resized such that its largest side is capped at 512 pixels, while preserving the original aspect ratio. This resizing step significantly reduces the total number of pixels processed during clustering, thereby lowering the computational cost of pixel-to-palette distance calculations and centroid updates.

Tools:

- Python
- Pillow / PIL (image loading + resizing)
- Runtime profiling

3.5.3 The existing algorithm can generate duplicate / near-duplicate palette colors due to lack of perceptual separation

Theory: During Stage 2 refinement, Fast K-Means updates each centroid using mean-based clustering. When an image is dominated by similar tones (e.g., large dark regions), multiple centroids may converge to very similar colors. Because the algorithm does not enforce a perceptual separation constraint, it can treat two numerically different RGB values as separate palette entries even if they are visually indistinguishable. This wastes palette slots and reduces the palette's ability to represent the image's true color variety.

Method: To prevent redundant palette entries, this study introduces a perceptual separation step after palette generation. Each palette color is converted from RGB to CIELAB, and pairwise perceptual distances are computed using ΔE. If any two palette colors fall below a minimum separation threshold (e.g., $\Delta E < \tau$), they are considered near-duplicates. One of the redundant colors is retained (typically the one representing the larger cluster), while the other palette slot is replaced with a candidate color selected to maximize its minimum ΔE distance from the existing palette. This ensures all K colors remain present while improving palette diversity and visual distinctness.

Tools:

- Python
- NumPy (pairwise checks, selecting replacement)
- scikit-image (CIELAB + ΔE computation)

3.6 Dataset Description

For this study, the USC-SIPI Image Database is used as the primary dataset for evaluating the proposed color palette generation algorithm. The database is a well-known and widely used collection of digitized images maintained by the Signal and Image Processing Institute (SIPI) at the University of Southern California. Its broad usage in

image processing research makes it a suitable benchmark for testing and comparing algorithmic performance in tasks related to color analysis, clustering, and representation.

LIST OF REFERENCES

Boyer-Moore Algorithm. (2025, March 25). https://www.tutorialspoint.com/data_structures_algorithms/boyer_moore_algorithm.htm

Charras, C., & Lecroq, T. (2004). Analysis of memory-efficient data structures in exact string matching. Handbook of Exact String Matching Algorithms, 1(1), 15–28. https://doi.org/10.1007/978-1-4020-8022-7_2

Crochemore, M., & Rytter, W. (2002). Optimizing completeness in text search through structured pattern detection. Jewels of Stringology: Text Algorithms, 2(1), 89–101. https://doi.org/10.1142/9789812777340_0005

Domingo, J. M. M., Martin, M. D. C., & Agustin, V. A. (2024). An enhancement of Boyer-Moore algorithm using Hash table. *International Journal for Multidisciplinary Research*, *6*(6). https://doi.org/10.36948/ijfmr.2024.v06i06.32548

Google Cloud. (2022). Ethereum Blockchain [Data set]. Kaggle. https://www.kaggle.com/datasets/bigquery/ethereum-blockchain

Malwa, S. (2024, December 30). Crypto scammers have finally found their thieving match: themselves. *CoinDesk*. https://www.coindesk.com/markets/2024/12/30/crypto-scammers-are-now-stealing-from-other-crypto-thieves

Manacher, G., Graham, S. L., Robert S. Boyer, Stanford Research Institute, J Strother Moore, & Xerox Palo Alto Research Center. (1977). *A fast string searching algorithm*.

Nair, V. (2025, May 16). *Best Solana Wallet Alerts: Track Wallet Activity | Ledger*. Ledger. https://www.ledger.com/academy/topics/crypto/solana-wallet-alerts

Navarro, G. (2001). An investigation of pattern shift behaviors in approximate string searching. ACM Computing Surveys, 33(1), 31–88. https://doi.org/10.1145/375360.375365

Purwanto, Y. S., Rifai, M. F., Jatnika, H., & Ardelia, G. A. (2022). Information Retrieval in Text-Based Document using Boyer Moore Algorithm. *JATISI (Jurnal Teknik Informatika Dan Sistem Informasi)*, *9*(2), 1308–1316. https://doi.org/10.35957/jatisi.v9i2.2053

Sernicula, L., Manalo, M., Niebre, G. A., Anastacio, R., Alagos, J., & Sermeno, J. (2023). Analysis of the enhanced Boyer-Moore search algorithm for a desktop PC search engine. *Journal of Innovative Technology Convergence.*, *5*(2), 45–56. https://doi.org/10.69478/jitc2023v5n2a05

Setianingrum, A. H., Rosydi, T., & Hermawan, R. (2020). Implementation of Boyer Moore Algorithm to Search Criminal Law Based on Criminal Code. *Atlantis Press*. https://doi.org/10.2991/assehr.k.200220.008

Teo, J., & Dobson, R. (2020, October 19). *Teaching computers to read health records is helping fight COVID-19 – here's how*. The Conversation. https://theconversation.com/teaching-computers-to-read-health-records-is-helping-fight-covid-19-heres-how-147385

Zha, X., Sahni, S., & Computer and Information Science and Engineering, University of Florida. (2010). GPU-to-GPU and Host-to-Host multipattern string matching on a GPU. In *Computer and Information Science and Engineering*. University of Florida. https://www.cise.ufl.edu/~sahni/papers/journal.pdf